Universal Optical Character Recognition (OCR) System

Comprehensive Guide: Streamlit Cloud Deployment, Docker Packaging & Customer Handover

Document Version: 1.0.0
Author: DevOps & Systems Architecture Team
Date: 2026-10-21
Status: Production Ready

---

1. Executive Overview & Delivery Strategy

The Universal Optical Character Recognition (OCR) & License Plate Detection System is architected to support two distinct delivery pathways tailored to the technical requirements and regulatory constraints of different customer personas:

- **Pathway 1 (End-Users & General Public): Zero-Install Web Delivery via Streamlit Community Cloud**
  Users interact with the AI application through a secure, public HTTPS hyperlink on any modern desktop, tablet, or smartphone web browser without downloading any software, installing Python, or configuring dependencies.

- **Pathway 2 (Enterprise, Logistics, Law Enforcement & High-Security Clients): Self-Contained Docker Containerization**
  Enables toll plaza operators, logistics warehouses, parking enforcement authorities, and corporate security teams to run the AI system 100% on-premise on private servers behind internal firewalls. This guarantees complete data confidentiality, zero external telemetry or cloud exposure, and uninterrupted offline operational continuity.

---

2. Streamlit Cloud Public Link Setup (Step-by-Step)

Streamlit Community Cloud provides free, continuous cloud hosting directly synchronized with a GitHub repository. Follow these exact steps to deploy or maintain the live production application:

Step 2.1: Repository Preparation on GitHub

The project repository must be public and contain the following essential files at the root level:

- `src/app.py`: The main entry point containing the Streamlit web dashboard interface.

- `requirements.txt`: Defines all pinned Python dependencies (`streamlit`, `easyocr`, `opencv-python-headless`, `pillow`, `numpy`, `pytest`) configured with the CPU-optimized PyTorch wheel index for lightweight, rapid builds.
- `packages.txt`: Specifies low-level Linux Debian packages (`libgl1`, `libglib2.0-0`) required for OpenCV headless image decoding.
- `models/model_manifest.json`: Documents the CRAFT text detector and CRNN recognition neural network architecture.
- `tests/test_images/`: Contains sample benchmark images for instant out-of-the-box customer evaluation.

Step 2.2: Launching on Streamlit Cloud

1. Navigate to https://share.streamlit.io and authenticate with your GitHub account.
2. Click the **"New app"** or **"Create app"** button in the top right corner.
3. Configure the three deployment parameters:
   - **Repository:** `pankajatd/ocr_detection_with_7SDLC_MultiAgents`
   - **Branch:** `main`
   - **Main file path:** `src/app.py`
4. Click **"Deploy!"**. Streamlit Cloud automatically allocates an isolated Linux container, installs system packages from `packages.txt`, installs Python packages from `requirements.txt`, and launches the service.

Step 2.3: Managing and Sharing Your Public Link

Once deployed, Streamlit assigns a permanent production URL:
Live URL: https://ocrdetectionwith7sdlcmultiagents-hepxuqwx3fzyi9xjbymdp8.streamlit.app/

Any Git commit pushed to the `main` branch automatically triggers an instant, zero-downtime redeployment.

---

3. Docker Containerization & Local Execution (Step-by-Step)

Docker eliminates the classic *"it works on my machine"* problem by bundling the application, exact Python 3.10 interpreter, low-level OS graphics drivers, and AI dependencies into an immutable, portable shipping container.

Step 3.1: Anatomy of the Dockerfile

Our production `Dockerfile` defines a minimal, hardened container image structured into six stages:

- **Base Operating System (`FROM python:3.10-slim`):** Initializes a stripped-down Debian Linux environment (~150MB) optimized for fast loading and low attack surface.

- **Low-Level System Libraries (`RUN apt-get install libgl1 libglib2.0-0`):** Installs Linux C++ shared libraries necessary for OpenCV matrix transformations without GPU dependencies.
- **Working Directory (`WORKDIR /app`):** Establishes a predictable, isolated sandbox directory.
- **Dependency Caching (`COPY requirements.txt .` followed by `RUN pip install`):** Uses Docker layer caching with `--extra-index-url https://download.pytorch.org/whl/cpu` so re-builds finish in seconds.
- **Application Ingestion (`COPY src/ models/ tests/`):** Copies application source code, neural network configurations, and verification test suites.
- **Network & Entrypoint (`EXPOSE 8501` & `ENTRYPOINT`):** Exposes port 8501 and launches Streamlit bound to address `0.0.0.0`.

Step 3.2: Why `docker-compose.yml` is Used

The `docker-compose.yml` file serves as a persistent configuration shortcut. It pre-configures: * Port mapping (`8501:8501`) * Container naming (`ocr_app`) * Restart policy (`unless-stopped`)

This allows operators to build, launch, and manage the complete containerized stack using single standard commands.

Step 3.3: How to Build and Run Docker

1. Ensure **Docker Desktop** is running on your host machine and shows **"Engine running"** (green indicator).
2. Open Command Prompt or PowerShell in the project directory:

   ```
   cd C:\Pankaja\AUTOMATED_SDLC_AGENT_PROJECTS\ocr_detection
   ```

3. Execute the single build-and-run command:

   ```
   docker-compose up -d --build
   ```

4. To stop the running container at any time:

   ```
   docker-compose down
   ```

---

4. How to Test and Verify Docker is Running

After executing `docker-compose up -d --build`, verify operational status using three independent verification methods:

Method A: Terminal Status Check

In Command Prompt or PowerShell, run:

```
docker ps
```

Look for the container named `ocr_app`. The status should show `Up X seconds` with port mapping `0.0.0.0:8501->8501/tcp`.

Method B: Browser Dashboard Test

Open Chrome, Edge, or Firefox and navigate to:
`http://localhost:8501`

The OCR Web Dashboard will render. You can test live text detection: 1. Select a sample test image (e.g., `1_motion_blur.jpg` or `3_skew_15deg.jpg`) or upload a custom image. 2. Confirm the detection pipeline generates bounding boxes with green overlays and outputs parsed alphanumeric strings with confidence percentages. 3. Verify the CSV export and annotated image download functionality.

Method C: Docker Desktop Visual Inspection

Open Docker Desktop and click the **"Containers"** tab. The `ocr_app` container entry will display a bright green active circle indicating it is healthy and actively serving requests.

---

5. Customer Handover & Delivery Protocols

Depending on customer requirements and security constraints, deliver the solution according to the following protocols:

Delivery Model A: Enterprise / Corporate Client (Docker Handover)

- **Target Audience:** Tollway concessionaires, logistics hub operators, corporate parking facilities, municipal traffic enforcement.
- **Delivery Package:** Provide access to the Git repository or deliver an encrypted `.zip` archive containing:
  - `Dockerfile` and `docker-compose.yml`
  - `src/` (`app.py`, `ocr_engine.py`, `utils.py`)
  - `models/` (`model_manifest.json`)
  - `requirements.txt`, `packages.txt`, and `README.md`
- **Customer IT Action:** The client extracts the package on their internal server and runs:

  ```
  docker-compose up -d --build
  ```

  The system runs securely on their local intranet (e.g., `http://192.168.1.100:8501`) with zero telemetry or cloud dependencies.

Delivery Model B: End-User / Evaluation Client (Public Web Hyperlink)

- **Target Audience:** Evaluators, executive stakeholders, remote field testers, product managers.

- **Delivery Package:** Share the permanent production web link: `https://ocrdetectionwith7sdlcmultiagents-hepxuqwx3fzyi9xjbymdp8.streamlit.app/`
- **Customer Action:** The user clicks the link on any browser. No technical setup, command prompt execution, or software installation is required.

---

6. Quick-Reference Handover Matrix

Parameter	Pathway 1: Cloud Deployment	Pathway 2: Enterprise On-Premise
Primary Platform	Streamlit Community Cloud	Docker & Docker Compose
Target Audience	General public, evaluators, mobile users	Enterprise IT, toll operators, logistics
Source Repository	`pankajatd/ocr_detection_with_7SDLC_MultiAgents`	Local Git clone or secure archive
Launch Command	Automated via `src/app.py`	`docker-compose up -d --build`
Access URL	`https://[app-name].streamlit.app`	`http://localhost:8501`
Data Privacy	Cloud isolated runtime	100% private on-premise, zero internet calls
Status Verification	Cloud dashboard logs	`docker ps` & `docker logs -f ocr_app`
Testing Suite	Automated GitHub Actions CI	`docker exec -it ocr_app pytest tests/`